
Cross-Model Spec Comparison: Finding Disagreement in Candidate Lean 4 Specifications Generated by OpenAI Models¹

Henry Ward
Independent Researcher

With
Apart Research

Abstract

We study whether candidate formal specifications produced by different OpenAI models agree on the intended behavior of small systems. For three specification tasks, we generate three Lean 4-oriented candidates per task, normalize them into a shared canonical format, and compare assumptions, preconditions, postconditions, invariants, and semantic boundary choices. We then search for concrete counterexamples using executable validators and property-based style witnesses. In the live OpenAI-backed run, the models converged on the sortedness and token-bucket tasks, but diverged sharply on token expiry: every pairwise comparison for that task produced a counterexample at the exact expiry boundary. The main takeaway is that cross-model disagreement is a practical signal for underspecification when the prompt exposes a crisp semantic boundary, and that a lightweight normalization plus witness-generation pipeline can turn that disagreement into reportable evidence under a very small budget.

¹ Research conducted at the [The Secure Program Synthesis Hackathon](#), May 2026

1. Introduction

AI systems can draft candidate formal specifications, but drafting a spec is not the same as validating that the spec captures the intended behavior. In practice, the difficult part is often hidden at the boundaries: inclusive versus exclusive thresholds, ordering constraints, or assumptions that were never written down. This matters for secure program synthesis because an underspecified or overconstrained specification can make later verification, synthesis, or review meaningless even when the generated artifact looks polished.

This project explores a simple hypothesis: if two language models independently generate candidate specs for the same task, the places where they disagree are likely to correspond to genuine ambiguity in the underlying requirement. We built a pipeline that asks two OpenAI models for Lean 4-centered spec candidates, normalizes their outputs into a shared schema, and then probes the disagreements with executable witnesses. The goal is not to prove semantic equivalence; it is to surface the places where a spec needs tightening.

Our main contributions are:

- A Lean 4-centered normalization and comparison pipeline for candidate specifications generated by two OpenAI models.
- A live evaluation on three tasks showing that the method is most informative when the boundary condition is sharp, with token expiry producing a counterexample in all nine pairwise comparisons.
- A reproducible reporting artifact, including a static HTML summary and a generated figure, that makes the disagreement pattern easy to inspect.

2. Related Work

This project builds on three strands of prior work. First, Lean 4 and its theorem-proving ecosystem show how informal requirements can be turned into mechanically checkable specifications and proofs, but they do not by themselves solve the upstream question of which spec to write [1]. Tooling that brings practical validation and assurance to Lean artifacts, such as the lean-tcb project, illustrates the value of integrating specification and validation workflows early in development [2].

Second, property-based testing and fuzzing demonstrate how executable witnesses can expose corner cases and hidden assumptions; modern Python tooling such as Hypothesis makes property-based exploration broadly accessible, while fuzzing tools (e.g., American Fuzzy Lop) highlight how automated input generation can find subtle boundary failures [3,4].

Third, recent discussions of specification brittleness emphasize that the hardest part of verification is often producing a precise, unambiguous requirement rather than mechanically checking a given one [5]. Our method differs from standard workflows by treating disagreement between independently generated candidate specs as the primary signal of underspecification, and by using lightweight normalization plus witness search to surface concrete examples of divergent behavior.

3. Methods

We evaluated three tasks: sortedness, token-bucket rate limiting, and token expiry semantics. For each task we requested three candidate specifications per model from ``gpt-4o`` and ``gpt-3.5-turbo``, using short JSON-only prompts that asked for a Lean 4-style spec fragment, assumptions, preconditions, postconditions, invariants, examples, and notes. The generation settings used a low temperature and a tight token budget so that the run could fit inside the hackathon constraints.

Each output was normalized into a canonical record with a ``semantic_mode`` tag and structured fields for assumptions, preconditions, postconditions, invariants, examples, and notes. For offline validation, the pipeline uses deterministic fixtures so the demo is reproducible; for live validation, it sends requests directly to the OpenAI API and parses the returned JSON. We then compare all cross-model pairs for each task and score them by structural difference and semantic mode disagreement.

To make the comparisons concrete, we search for a witness whenever the semantic modes differ. The witness generator is intentionally simple: for sortedness it uses a reordered duplicate pair, for rate limiting it uses a boundary-timestamp request, and for token expiry it uses a token checked exactly at the expiry instant. If one spec accepts the witness and the other rejects it, we record that as a counterexample.

The implementation produces both a static HTML report and a simple PDF summary. The report includes the normalized specs, the pairwise disagreement table, and a small generated figure summarizing how many pairwise comparisons produced counterexamples for each task.

4. Results

The live OpenAI-backed run generated 18 candidate specifications total: three tasks times three samples per model times two models. This yields 27 cross-model pairwise comparisons. The outcome was mixed: the sortedness task and the rate-limiter task converged under the prompts we used, while token expiry produced a crisp boundary disagreement.

Task	Pairwise comparisons	Counterexample -bearing comparisons	Best semantic split	Representative witness
Sortedness property	9	0	stable vs stable	No counterexample found
Token-bucket rate limiter	9	0	refill_first vs refill_first	No counterexample found
Token expiry semantics	9	9	exclusive vs inclusive	now == expiry_time

For token expiry, the representative witness is straightforward: issue time 100, expiry time 200, and now equal to 200. The inclusive interpretation accepts the token at the boundary, while the exclusive interpretation rejects it. That is exactly the kind of disagreement this method is meant to expose.

5. Discussion and Limitations

The strongest result here is also the most modest one: the pipeline reliably detects boundary ambiguity when the prompt exposes one, but it does not force divergence where the two models already agree. In this run, sortedness and token-bucket rate limiting converged across samples, which could mean either that the models genuinely aligned or that the prompts were not adversarial enough to push them apart. Either way, the result is useful because it tells us where the spec is already relatively settled and where it is still fragile.

The limitations are important. The task set is small, the sample size is small, and the method is sensitive to prompt design. The disagreement classifier is intentionally lightweight and can produce false positives or false negatives when outputs are verbose, inconsistent, or not perfectly JSON-shaped. More importantly, the method tests plausibility of a specification, not full semantic correctness. It cannot tell us whether a spec is globally right; it can only tell us when two independently generated candidates imply different behaviors.

There is also a dual-use angle. A tool that helps identify weak specs can also help an attacker or red teamer find the exact boundary where a spec breaks down. That risk is one reason we keep the demo centered on small, non-sensitive tasks and on the mechanics of validation rather than on any operationally dangerous domain.

Limitations

Although the live run produced a clear example of cross-model disagreement for the token expiry task, several limitations constrain how broadly these results can be interpreted. The evaluation covers only a very small set of tasks and uses just three samples per model, so the findings are indicative rather than statistically representative. Results are also sensitive to prompt design and generation settings: stronger, adversarial, or differently phrased prompts could surface different disagreement patterns or push models toward alternative semantic choices.

The witness search implemented here is intentionally lightweight and focused on obvious boundary cases; it does not perform exhaustive or adversarial exploration of the input space, and therefore may miss subtler semantic differences. Finally, the experiments were conducted on a tight budget and should be treated as evidence of the pipeline's usefulness in surfacing underspecification rather than as benchmark-grade measurements of model reliability or generalization.

Future Work

There are several clear directions to strengthen and scale this approach. First, expanding from three tasks to a larger, more diverse benchmark of specification boundary conditions would allow quantitative assessment of how often model disagreement corresponds to meaningful underspecification. Second, improving the normalization pipeline - for example by adding a more robust parser for non-JSON model outputs and richer Lean 4 canonicalization - would reduce noise introduced by formatting and surface-level differences.

Third, integrating property-based fuzzing and automated witness synthesis more deeply would transform the current, hand-seeded witness checks into a systematic counterexample generator capable of discovering less obvious divergences. Finally, extending the study to more models, experimenting with varied prompting strategies (including adversarial prompts), and measuring disagreement stability under different generation settings would provide a clearer picture of when and why this signal is most informative.

6. Conclusion

Cross-model spec comparison is a practical way to find where a candidate specification still has unresolved ambiguity. In our live run, the technique was most effective on token expiry, where the exact boundary condition produced a counterexample for every model pair, and less effective on the other two tasks, where the models converged. That is still a useful outcome: it shows where the spec is already stable and where it needs more precision.

The broader takeaway is that validation should not start after the spec is fixed. It should be part of the spec-writing loop itself, with model disagreement and concrete witnesses used as feedback to tighten the requirement before synthesis or proof work begins.

Code and Data

- Code repository: <https://github.com/kudossience/secure-program-synthesis>
- Live report artifacts: [../artifacts/live/report.html](#), [../artifacts/live/results.json](#), [../artifacts/live/figure1.svg](#)
- Offline reproducibility artifacts: [../artifacts/demo/report.html](#), [../artifacts/demo/results.json](#)

References

1. Lean 4 Reference Manual — The official Lean language reference and manual. <https://leanprover.github.io/reference/>
2. OathTech / Lean community — lean-tcb: Trusted computing base analyzer for Lean 4 (GitHub). <https://github.com/OathTech/lean-tcb>
3. Hypothesis — Property-based testing for Python (documentation). <https://hypothesis.readthedocs.io/>
4. American Fuzzy Lop (AFL) — A widely used coverage-guided fuzzing tool. <https://lcamtuf.coredump.cx/afl/>
5. Dodds, M. (2025). "Specifications Don't Exist" — Galois blog post on specification brittleness. <https://galois.com/blog/specifications-dont-exist/>
6. Apart Research (2026). Secure Program Synthesis Hackathon description and guidelines. <https://apartresearch.com/sprints/secure-program-synthesis-hackathon-2026-05-22-to-2026-05-24>

Appendix

Appendix A: Prompts Used

The prompts asked each model to return JSON only with a ‘semantic_mode’, a Lean 4-style spec fragment, assumptions, preconditions, postconditions, invariants, examples, and notes. The task text was kept short and fixed across samples so that model disagreement would reflect interpretation rather than prompt drift.

Appendix B: Canonical Schema

The canonical schema stores:

- semantic_mode
- lean
- assumptions
- preconditions
- postconditions
- invariants
- examples
- notes

This is enough to compare structure, surface the semantic boundary, and attach a concrete witness when the modes differ.

Appendix C: Additional Results

- Sortedness: 0/9 pairwise comparisons produced counterexamples.
- Token-bucket rate limiter: 0/9 pairwise comparisons produced counterexamples.
- Token expiry: 9/9 pairwise comparisons produced counterexamples.

LLM Usage Statement

We used OpenAI models to generate candidate specifications and to help draft initial planning text. All quantitative claims in the report were verified against saved artifacts produced by the pipeline, and the final report content was checked against the generated outputs.